

Kafka Cluster, Topics & Partitions

Broker, partition, replication and KRaft | One clear answer per concept

01 THE WORDS YOU MUST BE ABLE TO EXPLAIN

Broker	A Kafka server that stores partition replicas and serves client requests.
Topic	A named stream of related records, divided into partitions.
Partition	An ordered log of records that enables parallel processing.
Offset	A record's position within one partition; it is not a global event ID.
Replication factor	The total number of copies of each partition, including the leader.
ISR	In-sync replicas: the replicas Kafka considers caught up with the leader.

02 PARTITIONS DISTRIBUTE DATA; REPLICAS COPY IT

Illustration only: 3 partitions, replication factor 3. L = leader; F = follower.
Business-topic counts and leader placement must be checked at runtime.

Partition	Broker 1	Broker 2	Broker 3
P0	L: P0	F: P0	F: P0
P1	F: P1	L: P1	F: P1
P2	F: P2	F: P2	L: P2

3 partitions x RF 3 = 9 partition replicas, not 9 different data streams.

A producer writes to the partition leader; followers replicate its records.

03 ORDERING & PARALLELISM - FOLLOW-UP ANSWERS

Ordering: Kafka preserves log order within a partition, not across a topic.

Record key: with key-based partitioning and an unchanged partition count, the same key routes to the same partition.

Scale: more partitions allow more active consumers in one consumer group; extra consumers beyond the partition count stay idle in normal group assignment.

Kafka Cluster, Topics & Partitions

Your actual local setup | Network paths | Failure and durability questions

04 KRAFT - METADATA CONTROL, WITHOUT ZOOKEEPER

KRaft manages Kafka metadata through a controller quorum.

Our setup: Kafka 4.3.1; all three nodes run broker + controller roles.

Three controllers need a majority of two; one controller can be unavailable.

Windows-host applications

EXTERNAL: localhost + host ports

Docker application containers

INTERNAL: broker names + 19092

kop-kafka-1 | kop-kafka-2 | kop-kafka-3

Broker data traffic: 19092 | Controller quorum: 19093

05 LISTENERS - WHERE TO CONNECT

listeners: addresses and ports where the broker accepts connections.

advertised.listeners: addresses Kafka returns for clients to connect to.

Broker	Windows host	Inside Docker
1	localhost:9092	kop-kafka-1:19092
2	localhost:9094	kop-kafka-2:19092
3	localhost:9096	kop-kafka-3:19092

Why not localhost inside Docker? It points to that container itself.

Bootstrap servers provide initial discovery; advertised addresses must also work.

06 DURABILITY - WHAT IF A BROKER FAILS?

acks=all: producer waits for all current ISR replicas to acknowledge.

min.insync.replicas: minimum ISR size required for an acks=all write.

Example: RF=3, min ISR=2; writes can continue with two in-sync replicas.

If ISR falls to one, writes are rejected. Leader loss needs an eligible replacement.

Verified vs assumed: Compose sets internal offsets/transaction-topic RF=3 and transaction-topic min ISR=2. These do not set business-topic values. Order producer uses acks=all. Business-topic RF/min ISR need a runtime check. All containers share one laptop: this is not protection from laptop failure.

Interview scope: combined broker/controller roles suit this local setup; critical deployments should separate roles. Consumer ACK is a different concept from producer acks; we will cover it in the consumer chapter.

Sources: repository docker-compose.yml + README; Apache Kafka KRaft and topic configs.